

Pintos 2-2

(read, write, exec, wait, remove, filesize, seek, tell)

file descriptor : 파일을 식별하기 위한 번호

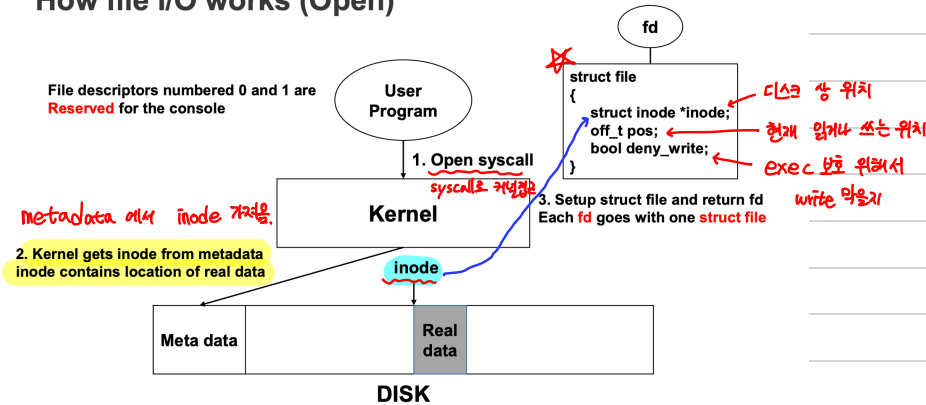
fd=0 (standard input) console 키보드 입력
 fd=1 (standard output) console 출력
 이 두개는 reserved for console

```
// read() 내부
if (fd == 0) {
    for (i = 0; i < size; i++)
        ((char *)buffer)[i] = input_getc(); // 키보드 입력
    return size;
}
```

```
// write() 내부
if (fd == 1) {
    putbuf(buffer, size); // 콘솔 출력
    return size;
}
```

• open

How file I/O works (Open)



2-1 내용이긴 함.

```
// 성공하면 새로운 파일 디스크립터(fd)를 반환, 실패하면 -1
int open(const char *file) {
    lock_acquire(&file_lock);
    struct file *f = filesystem_open(file);
    if (f == NULL) {
        lock_release(&file_lock); // 이거 안하면 deadlock
        return -1;
    }

    struct thread *cur_thread = thread_current();

    // fd = 0(STDIN_FILENO)은 표준 입력, fd = 1(STDOUT_FILENO)은 표준 출력, 실패하면 -1
    // fd_table index 늘리면서 file descriptor 리턴해야하니까 기록
    int fd = cur_thread->fd_idx++;

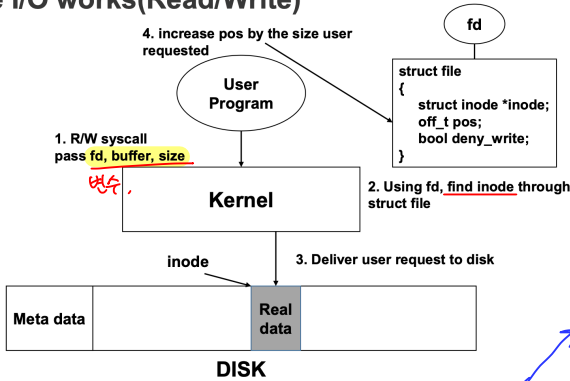
    if (fd >= FD_MAX) {
        printf("File descriptor table is full\n"); // 디버깅용
        file_close(f);
        lock_release(&file_lock);
        return -1;
    }

    cur_thread->fd_table[fd] = f; // 파일 저장

    lock_release(&file_lock);
    return fd;
}
```

• Read , Write

How file I/O works(Read/Write)



file_acquire → file_release
sema_down → sema-up

```

// ✓✓✓✓ - buffer가 valid한지 확인
bool is_valid_buffer(const void* buffer, unsigned size) {
    if(!buffer) return 0;
    char *ptr;
    struct thread *t = thread_current();
    for(ptr = (char *)buffer; ptr < (char *)buffer + size; ptr++) {
        if(!is_user_vaddr(ptr)) return 0; → PHYS_BASE 보다 작을까?
        if(!pagedir_get_page(t->pagedir, ptr)) return 0;
    }
    return 1;
}

```

byte 단위로 buffer 정제됨!
ptr 주소가 page table에 매핑되어있는가?

- `int read (int fd, void *buffer, unsigned size)`
 - Read the file opened with `fd` to `buffer`
 - Return the number of bytes read, or -1 on fail
 - Fd 0 reads from the keyboard using `input_getc()`

```

// ✓✓✓✓
// fd로부터 size만큼 데이터를 읽어 buffer에 저장
// fd == 0이면 키보드에서 입력, 그 외는 파일에서 읽음
// 읽은 바이트 수 반환, 실패 시 -1
int read(int fd, void *buffer, unsigned size) {

    // ✓✓✓✓ - buffer가 valid한지 확인
    if(!is_valid_buffer(buffer, size)) exit(-1);

    // fd == 0: 표준입력
    if (fd == 0) {
        int i;
        for (i = 0; i < size; i++) {
            // device/input.c의 input_getc()를 사용해서 키보드 입력을 받음
            ((char *)buffer)[i] = input_getc(); // 한 글자씩
        }
        return size;
    }

    // file descriptor check & file check
    if (fd < 2 || fd >= FD_MAX) return -1;
    struct file *f = thread_current()->fd_table[fd];
    if (f == NULL) return -1;

    // 파일 읽기
    lock_acquire(&file_lock);
    int bytes_read = file_read(f, buffer, size);
    lock_release(&file_lock);

    return bytes_read;
}

```

읽기, 쓰기 file_lock

- `int write (int fd, const void *buffer, unsigned size)`
 - Fd 1 writes to console: call `putbuf(buffer, size)` and return size, It should be implemented already for pintos 2-1.
 - Else, write size bytes from `buffer` to the file opened with `fd`, return size

```

// ✓✓✓✓ - fd에 size만큼 buffer 내용을 씴
// fd == 1이면 콘솔, 그 외에는 열린 파일에 기록
// 실제로 쓴 바이트 수를 반환 (실제로 더 적을 수 있음)
int write(int fd, const void *buffer, unsigned size) {
    // buffer가 valid한지 확인
    if(!is_valid_buffer(buffer, size)) exit(-1);

    // 콘솔에 출력
    if (fd == 1) {
        putbuf(buffer, size);
        return size;
    }

    // file descriptor check & file check
    if (fd < 2 || fd >= FD_MAX) return -1;
    struct file *f = thread_current()->fd_table[fd];
    if (f == NULL) return -1;

    // 파일에 출력
    lock_acquire(&file_lock);
    int bytes_write = file_write(f, buffer, size);
    lock_release(&file_lock);
    return bytes_write;
}

```

```

off_t
file_read (struct file *file, void *buffer, off_t size)
{
    off_t bytes_read = inode_read_at (file->inode, buffer, size, file->pos);
    file->pos += bytes_read;
    return bytes_read;
}

```

pos를 byte 수만큼 더함

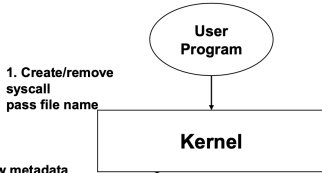
```

off_t
file_write (struct file *file, const void *buffer, off_t size)
{
    off_t bytes_written = inode_write_at (file->inode, buffer, size, file->pos);
    file->pos += bytes_written;
    return bytes_written;
}

```

• Create, Remove

How file I/O works(Create/Remove)



✗

2. - Create: setup new metadata
- Remove: delete metadata

metadata를 설정 / 삭제

DISK

• create: file을 initial size (byte)로 생성

```

// 성공하면 true, 실패하면 false
bool create(const char *file, unsigned initial_size) {
    if(!file) return 0;
    lock_acquire(&file_lock);
    bool ret = filesys_create(file, initial_size);
    lock_release(&file_lock);
    return ret;
}
  
```

생성만. 열려는건 open() 호출해야 함.

• Remove

• bool remove (const char *file)

- Remove a file with name, file
- If a file is open when it is removed, its blocks are not deallocated it may still be accessed by any threads that have it open, until the last one closes it

✗ remove된 파일이 아직 열려있다면??

metadata는 삭제되었지만,

struct file을 가지고있으면 접근 가능!

마지막 file_close() 후에 블록 반환 해제

• file remove: 열렸든 닫혔든 제거.

```

bool remove(const char *file) {
    if(!file) return 0;
    lock_acquire(&file_lock);
    bool ret = filesys_remove(file);
    lock_release(&file_lock);
    return ret;
}
  
```

메타 데이터만 삭제
(inode의 reference 문제)

① close 하는 경우

```

// 파일 디스크립터 (fd)를 닫기
// fd가 표준 입출력(STDIN, STDOUT)이면 무시
// fd가 범위 밖에 있으면 무시
// 유효한 fd라면 해당 파일을 닫고 fd_table 엔트리를 NULL로 초기화
void close(int fd) {
    if(fd < 2 || fd >= FD_MAX) return;
    struct thread *cur_thread = thread_current();
    file_close(cur_thread->fd_table[fd]);
    cur_thread->fd_table[fd] = NULL;
}
  
```

② close 안하고 process 종료

```

/* Free the current process's resources. */
void
process_exit(void)
{
    struct thread *cur = thread_current();
    uint32_t *pd;

    // ✓✓✓✓✓ - 자식 프로세스 종료할 때 부모를 깨움
    if (cur->parent != NULL) {
        sema_up(&cur->s_wait); // 부모가 wait() 중이면 깨워줌
    }

    // ✓✓✓✓✓ - 현재 실행중인 파일에 다시 쓰기가 가능하도록 바꾸음
    if (cur->cur_file) {
        file_allow_write(cur->cur_file);
        file_close(cur->cur_file);
        cur->cur_file = NULL;
    }

    /* Destroy the current process's page directory and switch back
     * to the kernel-only page directory. */
    pd = cur->pagedir;
    if (pd != NULL)
    {
        /* Correct ordering here is crucial. We must set
         * cur->pagedir to NULL before switching page directories,
         * so that a timer interrupt can't switch back to the
         * process page directory. We must activate the base page
         * directory before destroying the process's page
         * directory, or our active page directory will be one
         * that's been freed (and cleared). */
        cur->pagedir = NULL;
        pagedir_activate(NULL);
        pagedir_destroy(pd);
    }

    // ✓✓✓✓✓ - fd_table에 있는 파일들 모두 닫아줌
    int i;
    for (i = 2; i < FD_MAX; i++) {
        if (cur->fd_table[i]) {
            file_close(cur->fd_table[i]);
            cur->fd_table[i] = NULL;
        }
    }
}
  
```

현재 실행중인 파일
deny_write cant 종료
마지막 참조할 수 있는 4만큼.

모든 일반 파일 닫기

- `pid_t exec(const char *cmd_line)`
 - Executes `cmd_line` (program name and additional arguments, if any)
 - On success, returns new process's program id
 - On fail, return -1

• 자식 process 생성

```
// ~~~~~
// 새로운 사용자 프로세스를 실행하고 pid 반환
// 자식 프로세스가 load에 성공할 때까지 부모는 대기
// 실패 시 -1 반환
pid_t exec(const char *cmd_line) {
    tid_t tid = process_execute(cmd_line); // 새로운 스레드 생성
    if (tid == TID_ERROR) return -1;

    struct thread *child = get_child(tid);
    if (child == NULL) return -1;

    sema_down(&child->s_load); // 자식의 load() 완료까지 대기
    if (!child->is_load) return -1;

    return tid;
}
```

```
// ~~~~~
// 현재 스레드의 child_list 안에서 tid인 자식 프로세스를 찾아 리턴
struct thread *get_child(tid_t tid) {
    struct list_elem *e;
    struct thread *cur_thread = thread_current();

    for (e = list_begin(&cur_thread->child_list); e != list_end(&cur_thread->child_list); e = list_next(e)) {
        struct thread *child = list_entry(e, struct thread, child);
        if (child->tid == tid) {
            return child;
        }
    }
    return NULL; // 못찾음
}
```

thread 만으면서 list에 추가했던거 맞음.

• load는?

```
bool load(const char *file_name, void (**eip)(void), void **esp)
```

⇒ 실행파일 (ELF)를 user virtual memory에 적재

⇒ 프로그램 시작점, 초기 스택 설정

⇒ exec, start_process 중간단계를 성공실패 반환.

```
tid_t
process_execute(const char *file_name)
{
    char *fn_copy, *fn_parse, args;
    tid_t tid;

    fn_copy = palloc_get_page(0);
    fn_parse = palloc_get_page(0); // ~~~~~ - 파싱용으로도 따로 만들어줘야함
    if (fn_copy == NULL || fn_parse == NULL)
        return TID_ERROR;

    strcpy(fn_copy, file_name, PGSIZE);
    strcpy(fn_parse, file_name, PGSIZE);

    // 파싱용으로 따로 argument 버리고 전달
    char *program = strtok_r(fn_parse, " ", &args);

    /* Create a new thread to execute FILE_NAME. */
    tid = thread_create(program, PRI_DEFAULT, start_process, fn_copy);
    if (tid == TID_ERROR)
        palloc_free_page(fn_copy);

    palloc_free_page(fn_parse);

    return tid;
}
```

내부에서
현재 thread를 parent로
그리고 child list에
자식 추가

```
t->parent = thread_current(); // ~~~~~ - 자식의 parent 설정
list_push_back(&thread_current()->child_list, &t->child); // ~~~~~ - 자식 프로세스 리스트에 추가
```

start_process에서 파싱, load

```
// ~~~~~ - 파싱해서 스택에 삽입하는 로직
char *program, *args, *token;
program = strtok_r(file_name, " ", &args);
success = load(program, &if_.eip, &if_.esp);
```

+ 결과 부모에게 알려줌.

```
// ~~~~~ - 자식 프로세스의 load 성공 여부 저장
thread_current()->is_load = success;

// ~~~~~ - 부모 프로세스의 exec() 대기 해제 (load 완료 알림)
sema_up(&thread_current()->s_load);
```

• 새로운 thread 변수 정리

<code>thread->s_load</code>	exec 동기화를 세마포어 (부모 대기, 자식 깨움)
<code>thread->is_load</code>	자식 load 성공 여부를 부모가 확인하기 위한 flag
<code>thread->parent</code>	부모 자식 관계 유지
<code>thread->child_list</code>	부모가 소유한 자식 목록 저장

<code>sema_down()</code> → <code>sema_up()</code>	exec 동기화 핵심: 부모는 자식의 load가 끝날 때까지 기다려야 함
load 실패 시 <code>thread_exit()</code> 호출	자식이 바로 종료됨, 부모는 <code>wait()</code> 으로 상태 확인 가능
<code>file_deny_write()</code>	자식이 실행하는 실행파일에 대한 write 보호
<code>get_child(tid)</code>	부모의 <code>child_list</code> 안에서 tid 찾아줌, 자식 아닌 경우 NULL

• 예시 흐름.

```
User → exec("foo arg1 arg2") syscall
→ syscall_handler → exec()
→ process_execute()
→ thread_create() → start_process()
→ load() (file_deny_write)
→ is_load = true/false
→ sema_up(&s_load)
→ sema_down(&child->s_load)
→ return child_tid or -1
```

Wait

- `int wait (pid_t pid)`
- Please re-implement `wait()` following below instruction
 - wait for child process `pid` and retrieve the child's exit status
 - the child can already be terminated: return immediately
- If child was killed by kernel (failure), return -1
- If `pid` is not the direct child, return -1
 - even when `pid` is "grandson" of the caller
- If `wait` was called more than twice, return -1

```
int
process_wait (tid_t child_tid UNUSED)
{
    struct thread *child = get_child(child_tid);
    if (child == NULL) return -1;      // 자식이 아닌 경우
    if (child->is_wait) return -1;    // 이미 wait()한 경우

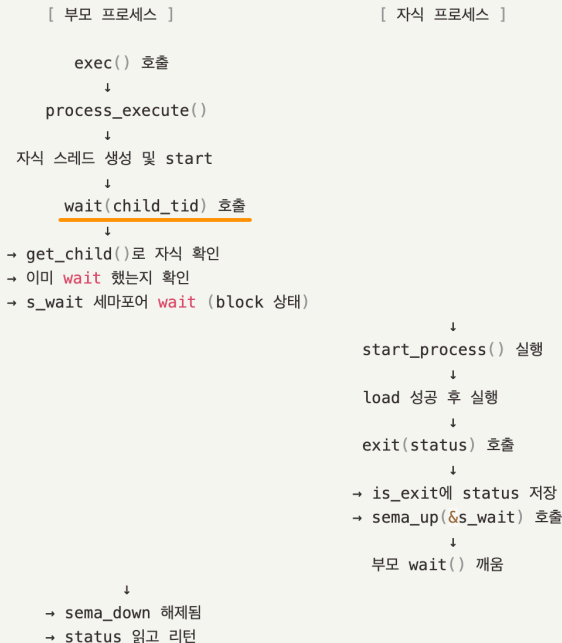
    child->is_wait = true;             // 중복 방지 플래그 설정
    sema_down(&child->s_wait);        // 자식이 죽을 때까지 대기

    int status = child->is_exit;       // 자식의 종료코드 획득
    list_remove(&child->child);       // child_list에서 제거
    return status;
}
```

```
void
process_exit (void)
{
    struct thread *cur = thread_current();
    uint32_t *pd;

    // 자식 프로세스 종료를 기다리는 부모를 깨움
    if (cur->parent != NULL) {
        sema_up(&cur->s_wait); // 부모가 wait() 중이면 깨워줌
    }
}
```

대기 해제



요구사항	구현 여부	설명
직계 자식만 기다릴 수 있어야 함	✓ <code>get_child(tid)</code> 에서 확인	
이미 wait한 자식은 다시 못 기다려야 함	✓ <code>is_wait</code> 플래그 체크	
자식이 이미 종료된 경우도 상태 반환해야 함	✓ 자식의 <code>is_exit</code> 저장값 사용	
부모가 wait 안한 채 먼저 종료해도 자식이 죽는 건 가능해야 함	✓ <code>process_exit()</code> 에서 조건 없이 <code>sema_up()</code> 호출	
wait() 후에는 자식의 <code>child_list</code> 에서 제거	✓ <code>list_remove(&child->child)</code> 수행	
자식이 예외로 죽은 경우 -1 반환	✓ <code>exit(-1)</code> 이 <code>is_exit</code> 로 전달됨	

🧠 주의 깊게 봐야 할 Race Condition 방지 포인트

- 동시 wait 방지 (`is_wait`)
 - 부모가 동일한 자식에게 두 번 wait하는 것을 차단
- 부모가 wait()하기 전에 자식이 죽는 경우
 - 자식이 먼저 `exit()` 호출해도, `sema_up()`을 호출하므로 부모가 나중에 `wait()` 해도 unblock됨
- 부모가 wait() 안 한 채 죽은 경우
 - 자식은 여전히 살아 있을 수 있지만 `thread_schedule_tail()`에서 `is_wait == true`일 경우에만 해제되게 되어 있음
 - 이 부분은 Project 2에서는 크게 신경 안 써도 됨

filesize

```
// ✓✓✓✓✓
// 열려 있는 파일 디스크립터 fd의 총 바이트 크기를 반환
// 잘못된 fd이거나 닫힌 파일이면 -1 반환
// 내부적으로 file_length() 사용
int filesize(int fd) {
    // file descriptor check & file check
    if (fd < 2 || fd >= FD_MAX) return -1;
    struct file *f = thread_current()->fd_table[fd];
    if (f == NULL) return -1;

    return file_length(f);
}
```

→ inode의 실제 byte 길이 반환

• seek : read, write 전에 원하는 위치로 이동

```
// ✓✓✓✓✓ - fd 파일에서 현재 보고있는 위치를 position으로 변경
void seek(int fd, unsigned position) {
    if (fd < 2 || fd >= FD_MAX) return;
    struct file *f = thread_current()->fd_table[fd];
    if (!f) return;
    file_seek(f, position);
}
```

```
struct file {
    struct inode *inode;
    off_t pos;           // ← 이게 현재 읽기/쓰기 포지션
    bool deny_write;
};
```

• tell : 현재 위치 조회 (값을 반환)

```
// ✓✓✓✓✓ - fd 파일에서 현재 보고있는 위치를 반환
unsigned tell(int fd) {
    if (fd < 2 || fd >= FD_MAX) return -1;
    struct file *f = thread_current()->fd_table[fd];
    if (!f) return -1;
    return file_tell(f);
}
```

```
/* Sets the current position in FILE to NEW_POS bytes from the
   start of the file. */
void
file_seek (struct file *file, off_t new_pos)
{
    ASSERT (file != NULL);
    ASSERT (new_pos >= 0);
    file->pos = new_pos;
}

/* Returns the current position in FILE as a byte offset from the
   start of the file. */
off_t
file_tell (struct file *file)
{
    ASSERT (file != NULL);
    return file->pos;
}
```

```
int fd = open("file.txt"); // file->pos = 0

read(fd, buf, 3); // 읽음: "abc"
printf("%d\n", tell(fd)); // → 3

seek(fd, 5); // file->pos = 5

read(fd, buf, 2); // 읽음: "fg"
printf("%d\n", tell(fd)); // → 7

seek(fd, 0); // file->pos = 0

read(fd, buf, 4); // 읽음: "abcd"
printf("%d\n", tell(fd)); // → 4
```

012345
file.txt ⇒ "abcdefjhij" (10 bytes)

작동 예시

abc
3
fg
5

file 관리

syscall_handler() get_user(f+esp) = SYS_EXEC
↓
exec()
↓
process_execute()
↓
thread_create()
↓
start_process()
↓
load() 중간에 deny_write

```
// ✓✓✓✓✓ - 현재 실행중인 파일에 쓰기 금지  
t->cur_file = file;  
file_deny_write(t->cur_file);
```

```
/* Free the current process's resources. */  
void  
process_exit (void)  
{  
    struct thread *cur = thread_current ();  
    uint32_t *pd;  
  
    // ✓✓✓✓✓ - 자식 프로세스 종료를 기다리는 부모를 깨움  
    if (cur->parent != NULL) {  
        sema_up(&cur->s_wait); // 부모가 wait() 중이면 깨워줌  
    }  
  
    // ✓✓✓✓✓ - 현재 실행중인 파일에 다시 쓰기 가능하도록 바꿔줌  
    if (cur->cur_file) {  
        file_allow_write(cur->cur_file);  
        file_close(cur->cur_file);  
        cur->cur_file = NULL;  
    }  
}
```

↪ 쓰기 가능하게
↪ file 닫기

```
/* Closes FILE. */  
void  
file_close (struct file *file)  
{  
    if (file != NULL)  
    {  
        file_allow_write (file);  
        inode_close (file->inode);  
        free (file);  
    }  
}
```

↪ open_cnt 감소.
↪ file 구조체 메모리 해제